OII462 · Tarea 6

Encoder estático, complejidad y variantes del Transformer para el WTAP

Informe: rediseño de la arquitectura, complejidad temporal, variantes y comparación experimental (puntos 1, 2, 3, 5 y 6)

[Nombre(s) del grupo] · Optimización combinatoria · 2026

Resumen. El Transformer con puntero del módulo 2 recibía en cada paso una secuencia de 1 + W·T filas con 16 features, de las cuales 12 dependen del estado; por eso su "encoder" se recalculaba en cada una de las W decisiones, con un costo cuadrático en el número de pares por paso. Lo rediseñamos con responsabilidades separadas: un encoder bipartito armas ↔ objetivos que solo ve V_j y p_{ij} (token aprendible para las armas y la matriz como aristas) y que corre una vez por instancia, y un decoder que integra el estado y es lineal en n = W·T por paso. Sobre esa base proponemos dos variantes: **A**, con sesgo inductivo (de salida y atencional) basado en la ganancia marginal de MMR, y **B**, que re-contextualiza los pares en cada paso con un bloque ISAB de costo lineal. Una decisión de 50×40 pasa de 24,0 ms a 0,65 ms. Con la misma configuración y 3 semillas por modelo, los tres superan a MMR desde la primera época de entrenamiento. En las 50 instancias de prueba 10×8 logran gaps de 2,33 % (base), 2,44 % (A) y 2,21 % (B), frente a 7,68 % de MMR y 2,97 % del módulo 2. Entrenados solo con 10×8, mejoran al módulo 2 en todos los tamaños que comparten (hasta 30×20) y a MMR hasta 50×40; en 100×80, MMR (1,23 %) queda mejor en promedio. A parte desde la política de MMR, y B obtiene la menor pérdida de validación y más soluciones iguales a las del experto, pero cada época le cuesta 4,2 veces más. Las diferencias de gap medio entre las tres arquitecturas son del orden de la variación entre semillas.

1. Rediseño de la arquitectura (punto 1)

1.1 Diagnóstico del modelo del módulo 2

En el módulo 2 la entrada era una matriz (1 + W·T) × 16: una fila global y una fila por par (arma i → objetivo j). El bloque de self-attention de 2 capas y el puntero se ejecutaban juntos en cada paso, porque la matriz se recalculaba con state2vec tras cada asignación. Recorriendo una construcción completa verificamos qué columnas cambian entre pasos:

Tipo	Features del módulo 2
Estáticas (4)	p_ij, valor, p_vs_mejor, es_global
Dinámicas (12)	no_disponible, asignado, q_sobrevive, valor_en_juego, ganancia, es_mmr, armas_en_obj, ventaja, arrepentimiento, mmr_completa, q_final_mmr, frac_restantes

Ese "encoder" no cumplía la definición de la tarea: dependía del estado y no podía ejecutarse una sola vez. Su costo por paso era la self-attention sobre 1 + W·T filas, $O((WT)^2)$ por capa, más state2vec, que es $O(W^2T)$ porque mmr_completa simula MMR hasta el final.

1.2 Arquitectura base rediseñada

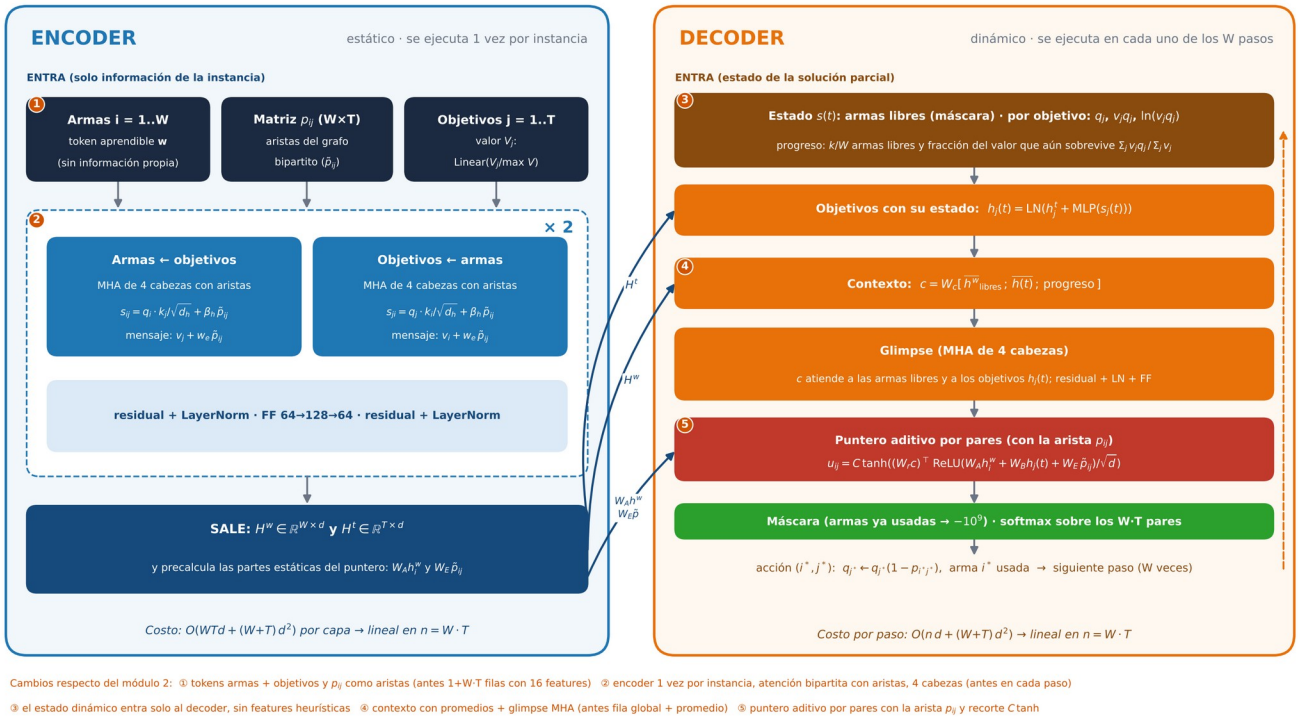


Figura 1. Arquitectura base rediseñada. A la izquierda, el encoder estático (1 vez por instancia); a la derecha, el decoder, que integra el estado en cada uno de los W pasos. Los círculos naranjos marcan los cambios respecto del módulo 2.

Encoder (estático). Las armas no tienen información propia: solo su fila de la matriz p. Siguiendo la sugerencia del enunciado, cada arma parte de un **token aprendible** compartido y cada objetivo de un embedding lineal de V_j/max V. La matriz entra como **aristas de un grafo bipartito**: en cada una de las 2 capas, las armas atienden a los objetivos y los objetivos a las armas (en paralelo, MHA de 4 cabezas), y la arista $\tilde{p}_{ij}$ se inyecta en el puntaje (un peso por cabeza) y en el mensaje (un vector aprendido), como en la EGAT del material y en MatNet (Kwon et al., 2021). Tras cada atención hay residual + LayerNorm y un feed-forward 64 → 128 → 64. La salida son los embeddings H^w (W×d) y H^t (T×d) y las partes del puntero que no dependen del estado, que se precalculan una vez.

$$s_{ij}^{(h)} = \frac{(W_Q h_i^w)^{(h)} \cdot (W_K h_j^t)^{(h)}}{\sqrt{d_h}} + \beta^{(h)} \tilde{p}_{ij}, \qquad m_i = W_O[\sum_j \mathrm{softmax}_j(s_{ij}^{(h)})\,((W_V h_j^t)^{(h)} + w_e^{(h)} \tilde{p}_{ij})]_h$$

Decoder (dinámico). Recibe el estado de la solución parcial: la máscara de armas libres y, por objetivo, q_j, $v_j q_j$ y $\ln(v_j q_j)$. Suma un MLP de ese estado al embedding de cada objetivo, arma un contexto con el promedio de las armas libres, el de los objetivos y el progreso, hace un glimpse (MHA de 4 cabezas, Kool et al., 2019) y puntúa cada par con un **puntero aditivo** que también ve la arista:

$$u_{ij} = C\tanh((W_r c)^\top \mathrm{ReLU}(W_A h_i^w + W_B h_j(t) + W_E \tilde{p}_{ij})/\sqrt{d}), \quad u_{ij} = -10^9 \text{ si } i \text{ ya fue usada}$$

El modelo ya no recibe features heurísticas (ganancia, arrepentimiento, anticipación con MMR): debe aprenderlas de V, p y q. Así la información dinámica entra solo al decoder y el encoder cumple la definición de la tarea.

1.3 Cambios respecto del módulo 2

#	Aspecto	Módulo 2	Base rediseñada
1	Tokens de entrada	1 + W·T filas (fila global + pares) con 16 features	W armas (token aprendible) + T objetivos (V_j) + matriz p_ij como aristas
2	Encoder	self-attention entre las 1 + W·T filas, 1 cabeza, se ejecuta en cada paso	atención cruzada armas ↔ objetivos con aristas, 4 cabezas, 1 vez por instancia
3	Información dinámica	dentro de las features de entrada (12 de 16)	solo en el decoder: máscara, q_j, valor en juego y progreso; sin features heurísticas
4	Contexto del decoder	[h_global ; promedio de filas disponibles]	[promedio de armas libres ; promedio de objetivos con estado ; progreso] + glimpse MHA
5	Puntero	q·h_r/√d sobre las filas	aditivo sobre pares, con la arista p_ij y recorte C·tanh
	Entrenamiento	60 000 estados independientes	teacher forcing: 1 encoder + W pasos del decoder por instancia

2. Complejidad temporal del encoder y del decoder (punto 2)

Sea n = W·T el tamaño de la matriz p, que es también el número de acciones; d = 64 la dimensión, L = 2 las capas y m = 16 los puntos inductores de la variante B. Un decoder que entrega un logit por acción necesita al menos Ω(n) por paso, así que ese es el mínimo alcanzable.

Modelo	Encoder (1 vez por instancia)	Cómputo por paso del decoder	Por instancia (W pasos)
Módulo 2	— (todo se recalcula en cada paso)	state2vec $O(W^2T)$ + self-attention $O(L \cdot n^2 \cdot d)$	$O(W \cdot n^2 \cdot d)$
Base	$O(L \cdot (n \cdot d + (W+T) \cdot d^2))$	$O(n \cdot d + (W+T) \cdot d^2)$ → lineal	$O(W \cdot n \cdot d)$
A · sesgo MMR	igual a la base + O(L·n)	base + O(n) (ganancias) → lineal	$O(W \cdot n \cdot d)$
B · ISAB	igual a la base	$O(n \cdot m \cdot d + n \cdot d^2)$ → lineal	$O(W \cdot n \cdot m \cdot d)$
B con self-attention completa	igual a la base	$O(n^2 \cdot d)$ → cuadrático	$O(W \cdot n^2 \cdot d)$

El encoder rediseñado es incluso lineal en n, porque la atención es bipartita (cada arma mira T objetivos y cada objetivo W armas), aunque la tarea permitiría uno cuadrático. El decoder de la base y de la variante A es lineal, así que no necesita un mecanismo de atención eficiente. La única pieza cuadrática que aparecería en el decoder es la interacción entre pares de la variante B; por eso la implementamos con ISAB (sección 3.2).

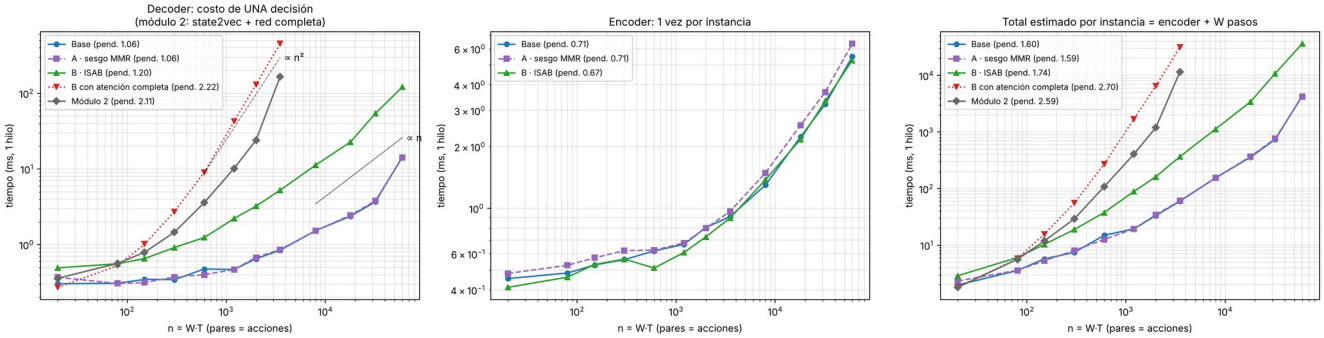


Figura 2. Tiempos medidos con PyTorch en 1 hilo (mediana de varias repeticiones, pesos aleatorios). Izquierda: una decisión; centro: el encoder; derecha: total estimado por instancia (encoder + W pasos). Pendientes log-log ajustadas con los 4 tamaños más grandes de cada curva.

Modelo	Pendiente por paso	Pendiente del encoder	Pendiente por instancia	Paso en 10×8	Paso en 50×40
Módulo 2	2,11	—	2,59	0,56 ms	24,04 ms
Base	1,06	0,71	1,60	0,31 ms	0,65 ms
A · sesgo MMR	1,06	0,71	1,59	0,31 ms	0,68 ms
B · ISAB	1,20	0,67	1,74	0,56 ms	3,22 ms
B con atención completa	2,22	0,20	2,70	0,53 ms	130,78 ms

Lectura. Las pendientes confirman el análisis: el paso del módulo 2 y el de B con self-attention completa crecen como n² (pendientes 2,11 y 2,22), mientras que el de los tres modelos nuevos crece de forma aproximadamente lineal (pendientes 1,06, 1,06 y 1,20). Lo que excede a 1 viene del tamaño más grande (300×200, n = 60.000), probablemente porque allí los tensores de un paso dejan de caber en la caché del procesador: sin ese punto, las pendientes son 0,65, 0,66 y 1,02. Con el mismo criterio, las del módulo 2 y de B con atención completa quedan en 1,48 y 2,06, todavía claramente superlineales. El encoder queda bajo 1 (pendientes 0,67–0,71) porque en los tamaños medidos todavía pesa el costo fijo de lanzar las operaciones. Cuesta 0,48 ms en 10×8 y se paga una sola vez. Por instancia, la base es 1,6 veces más rápida que el módulo 2 en 10×8 y 36 veces en 50×40.

3. Variantes propuestas (punto 3)

3.1 Variante A — sesgo inductivo con el criterio de MMR

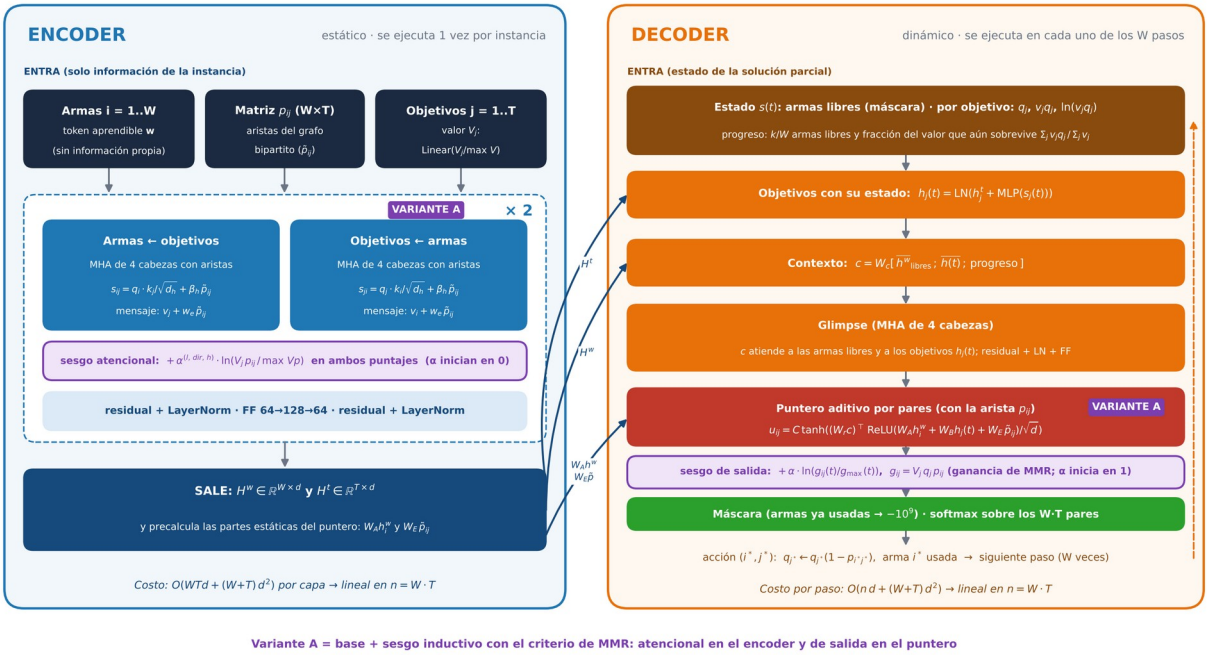


Figura 3. Variante A: la base más el sesgo inductivo (en morado), atencional en el encoder y de salida en el puntero.

Aplicamos las dos formas de sesgo inductivo vistas en el material, ambas con la ganancia marginal de MMR, g_ij = V_j·q_j·p_ij, y con pesos aprendibles para que el modelo decida cuánto confiar en el sesgo:

- **Sesgo de salida (decoder, dinámico):** u_ij ← u_ij + α·ln(g_ij(t)/g_max(t)), con α aprendible que parte en 1. La política queda π(i, j) ∝ e^u_ij·(g_ij/g_max)^α: con u = 0 es un "MMR suave" y con α → ∞ es MMR exacto, así que la red aprende correcciones sobre MMR. Como inicializamos W_r = 0 en los tres modelos, la variante A parte exactamente desde la política de MMR.
- **Sesgo atencional (encoder, estático):** en cada capa, dirección y cabeza se suma α^(l,dir,h)·ln(V_j p_ij / max Vp) al puntaje armas ↔ objetivos (la ganancia de MMR con la asignación vacía), con α^(l,dir,h) aprendibles que parten en 0, como en el material.

¿Por qué MMR es un criterio robusto? (i) Es la heurística constructiva clásica del WTAP (den Broeder et al., 1959) y es óptima cuando todas las armas son iguales: si p_ij = p_j el problema es una asignación separable y convexa, donde elegir siempre el mayor retorno marginal es óptimo. (ii) Es la construcción inicial de métodos del estado del arte (Ahuja et al., 2007; ver también la revisión de Kline et al., 2019) y de nuestro propio experto ILS. (iii) En nuestros datos produce soluciones muy superiores al promedio: gap 6,94 % frente a 290,3 % del constructivo aleatorio en 100 instancias de validación, y la acción del experto está casi siempre entre las de mayor ganancia:

Posición de la acción del experto en el ranking de MMR	top-1	top-2	top-3	top-5	top-10
% de las 3 000 decisiones de validación	79,9 %	96,3 %	99,1 %	100,0 %	100,0 %

Es decir, MMR acierta la decisión exacta del experto en ≈80 % de los casos y la tiene entre sus 3 mejores en ≈99 %: el sesgo empuja al modelo hacia una región muy buena de acciones y le deja aprender cuál elegir dentro de ella.

3.2 Variante B — decoder con atención eficiente (ISAB)

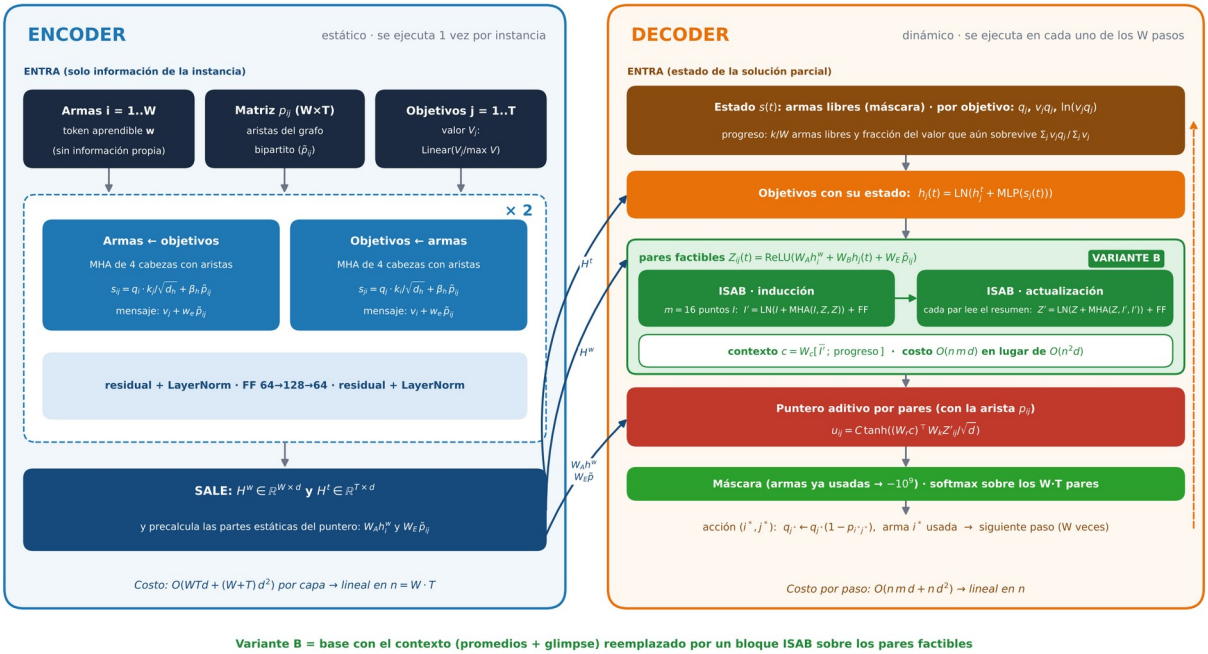


Figura 4. Variante B: el contexto de la base (promedios + glimpse) se reemplaza por un bloque ISAB sobre los pares factibles (en verde).

En la base, los pares solo compiten en el softmax final: un par no "sabe" si otra arma libre aprovecharía mejor su objetivo, información que en el módulo 2 aportaban el arrepentimiento y la self-attention entre filas. La variante B re-contextualiza el estado en cada paso: los pares factibles Z_ij(t) interactúan mediante un Induced Set Attention Block (Lee et al., 2019) con m = 16 puntos inductores aprendibles. Primero los puntos resumen a los pares, I' = LN(I + MHA(I, Z, Z)) + FF; luego cada par lee ese resumen, Z' = LN(Z + MHA(Z, I', I')) + FF. El contexto pasa a ser el resumen inducido, c = W_c[promedio de I' ; progreso], y el puntero usa los pares contextualizados.

Justificación del mecanismo. Una self-attention completa entre los n pares costaría O(n²·d) en cada paso: es justamente el costo del módulo 2 y rompe la exigencia de un decoder lineal (Figura 2, curva roja). ISAB la reemplaza por dos atenciones cruzadas de costo O(n·m·d). Lo preferimos a LUNA porque el estado del WTAP es un conjunto sin orden que cambia en cada paso (desaparecen los pares del arma asignada): no hay una secuencia ni una memoria que deba persistir entre capas, que es lo que aporta el depósito de LUNA, mientras que ISAB es el bloque del Set Transformer para conjuntos y construye un resumen nuevo en cada capa. La atención dispersa requeriría un criterio heurístico para decidir qué pares conectar, que es el papel del sesgo de la variante A; así cada variante aísla un efecto.

4. Configuración experimental (punto 4)

Hiperparámetro	Valor (igual para los tres modelos)
d / cabezas / capas del encoder / FF	64 / 4 / 2 / 64 → 128 → 64
Normalización · recorte de logits	LayerNorm post-residual · C = 10
Inicialización del puntero	W_r = 0 (los logits parten en 0)
Datos	6.000 instancias 10×8 de entrenamiento (60.000 decisiones), 300 de validación (3.000); mismas semillas que en las tareas anteriores
Optimizador	Adam, lr = 0.001; ReduceLROnPlateau (factor 0,5, paciencia 3, mínimo 1e-05), como sl_train del material
Batch · recorte de gradiente	32 instancias (320 decisiones) · norma 1,0
Épocas · early stopping	máx. 60 · paciencia 8 sobre la pérdida de validación (restaura la mejor época)
Pérdida	entropía cruzada sobre los W·T logits enmascarados (teacher forcing)
Semillas	3 (inicialización y orden de los batches); los datos son los mismos
Específicos de cada variante	A: α inicia en 1 y α^(l,dir,h) en 0 · B: m = 16 puntos inductores
Parámetros entrenables	Base: 192.656 · A · sesgo MMR: 192.673 · B · ISAB: 227.152

5. Convergencia (punto 5)

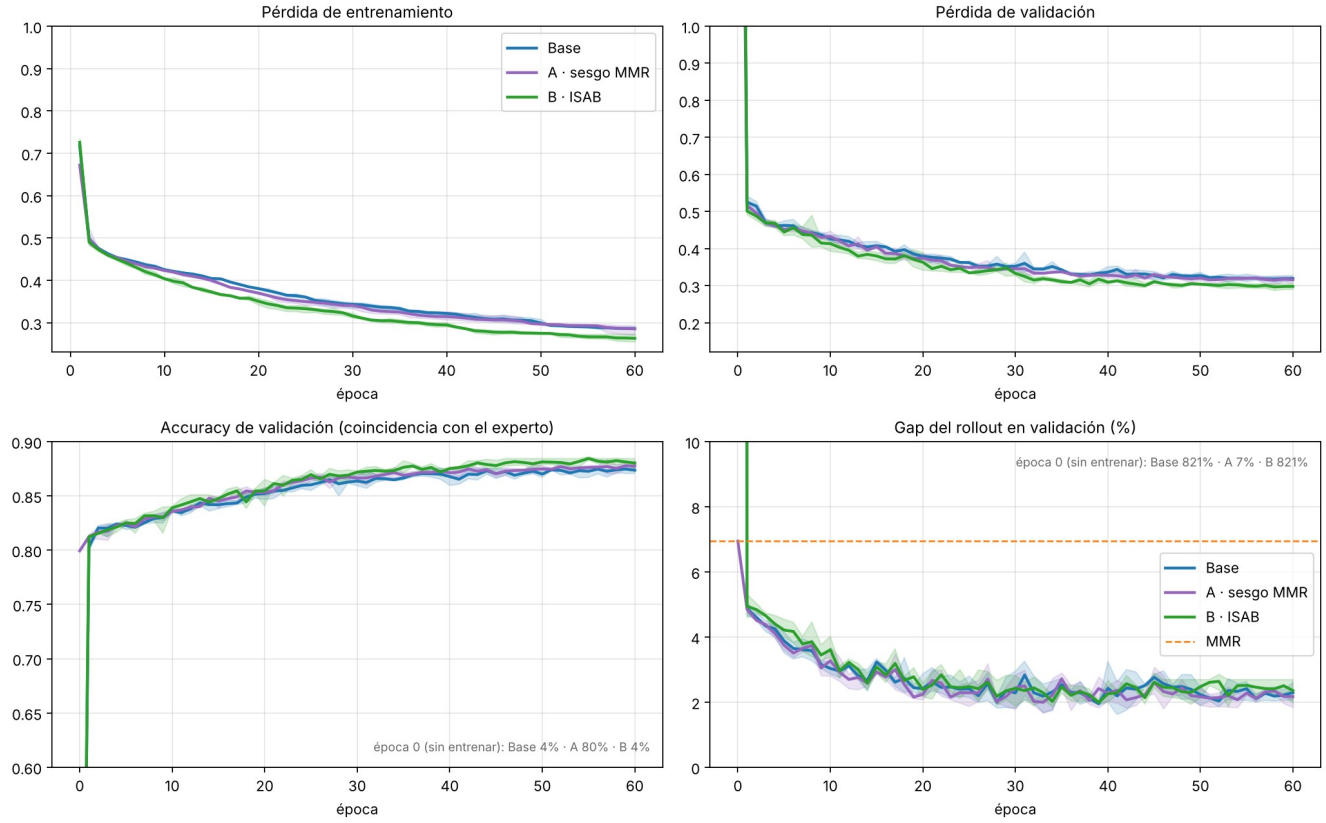


Figura 5. Curvas por época (línea = media de 3 semillas; banda = mínimo–máximo). La época 0 es el modelo sin entrenar; el gap de validación se mide con el rollout greedy en 100 instancias.

Modelo	Gap val. época 0	Épocas hasta gap < 3 %	Épocas hasta pérdida < 0,35	Mejor época	Mejor pérdida val. (media ± desv.)	Oscilación del gap (pp)	s / época	Minutos hasta gap < 3 %
Base	821,0 %	10–13	26–32	46–53	0,317 ± 0,006	0,22	3,6	0,7
A · sesgo MMR	6,9 %	9–12	25–28	44–58	0,311 ± 0,007	0,29	3,7	0,7
B · ISAB	821,0 %	11–14	20–24	38–58	0,293 ± 0,005	0,29	15,1	3,2

Épocas: rango entre las semillas. Oscilación del gap: promedio de |Δ gap de validación| entre épocas consecutivas en las últimas 10 épocas, en puntos porcentuales. Minutos: tiempo de reloj medio hasta el primer gap de validación < 3 % (CPU, incluye la validación de cada época).

Punto de partida. Sin entrenar, la base y B tienen logits constantes (W_r = 0), así que el rollout greedy asigna todas las armas al primer objetivo (gap 821 %). Gracias al sesgo de salida, A parte exactamente desde la política de MMR (gap 6,9 % y accuracy 80 %).

Velocidad. La ventaja inicial de A dura una época: tras la primera, los tres modelos ya superan a MMR (gap de validación 4,6–5,3 %) y las curvas de la base y de A casi coinciden (gap medio de las épocas 1–10: 3,90 % y 3,88 %; B 4,19 %). Medida en épocas, A llega a un gap < 3 % en 9–12, la base en 10–13 y B en 11–14. En la pérdida el orden cambia: B es la que baja más rápido (pérdida < 0,35 en 20–24 épocas, contra 25–28 de A y 26–32 de la base) y la que termina más abajo, con la mayor accuracy de validación (88,0–88,5 % frente a 87,3–87,9 %). En tiempo de reloj, en cambio, una época de B cuesta 4,2 veces más (15,1 s frente a 3,6–3,7 s): necesita 3,2 min para llegar a un gap < 3 %, contra 0,7 min de la base y de A, y sus 3 semillas tomaron 42 min de entrenamiento, contra 10–11 min.

Estabilidad. Las 9 corridas convergen sin divergencias: la mejor pérdida de validación varía poco entre semillas (desviación 0,005–0,007) y, en el tramo final, la pérdida oscila ≈ 0,005 entre épocas. El gap del rollout es más ruidoso (0,22–0,29 pp entre épocas consecutivas; la base es la más estable) porque depende de unas pocas decisiones críticas por instancia. Por lo mismo, la menor pérdida de B no se traduce en un menor gap de validación: los tres terminan en niveles parecidos (media de las últimas 10 épocas: base 2,28 %, A 2,19 % y B 2,35 %). El early stopping se activó en 4 de las 9 corridas; las otras 5 llegaron al máximo de 60 épocas con la mejor época entre la 53 y la 58, es decir, todavía mejoraban levemente. Como el presupuesto es el mismo para los tres modelos, la comparación es justa.

Pesos α de la variante A (Figura 6). El peso del sesgo de salida crece de 1 a 2,0–3,4 y 14 de los 16 pesos del sesgo atencional (semilla 0) terminan positivos; el mayor (1,94) está en la capa 1, en la atención de cada objetivo sobre las armas. El modelo usa el criterio de MMR en el encoder y en el decoder, y le da cada vez más peso.

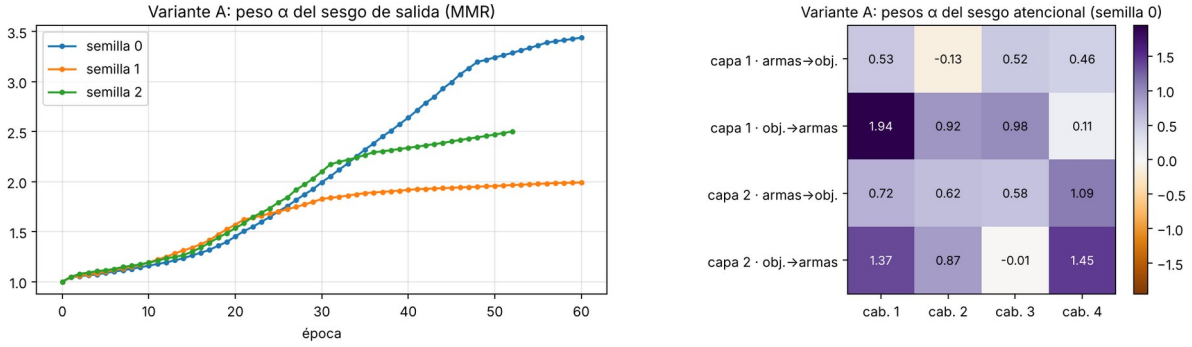


Figura 6. Variante A: evolución del peso α del sesgo de salida y pesos del sesgo atencional aprendidos (semilla 0).

6. Evaluación con instancias de distintos tamaños (punto 6)

Los modelos se entrenaron solo con 10×8 y se evalúan sin cambios en **7 tamaños**, con las mismas instancias del proyecto del módulo 2 (30 por tamaño; las 50 de prueba en 10×8), de modo que la columna "Módulo 2" es directamente comparable; 50×40 y 100×80 son nuevos. La referencia es el experto ILS de las tareas anteriores (óptimo en todas las instancias verificadas de 5×4 y 10×8): gap = 100·(costo/costo ILS − 1). Se evalúan los 3 modelos entrenados de cada arquitectura (una por semilla) con el rollout greedy, que verificamos idéntico al GreedyAgent de eda; la tabla reporta la media y la desviación estándar sobre las 3 semillas × las instancias de cada tamaño, y más abajo se muestra la variación entre semillas.

Tamaño	W·T	Base	A · sesgo MMR	B · ISAB	MMR	Aleatorio	Módulo 2
5×4	20	**0,71 ± 2,61**	1,00 ± 3,36	1,30 ± 3,99	6,46 ± 7,87	259 ± 102	1,67 ± 4,21
10×8	80	2,33 ± 2,93	2,44 ± 2,96	**2,21 ± 3,13**	7,68 ± 4,91	316 ± 100	2,97 ± 3,68
15×10	150	**3,13 ± 2,57**	3,40 ± 2,74	3,25 ± 3,03	7,82 ± 4,92	454 ± 160	5,28 ± 3,87
20×15	300	**1,96 ± 1,69**	2,09 ± 1,58	2,01 ± 1,86	5,71 ± 3,11	373 ± 90	3,99 ± 3,05
30×20	600	**2,95 ± 1,87**	2,97 ± 1,98	3,43 ± 1,90	5,89 ± 2,99	588 ± 104	5,30 ± 2,89
50×40	2000	2,01 ± 1,00	1,96 ± 1,04	**1,86 ± 1,03**	2,46 ± 0,92	430 ± 65	—
100×80	8000	1,57 ± 0,79	1,52 ± 0,60	**1,30 ± 0,71**	1,23 ± 0,53	452 ± 30	—

Gap vs el experto ILS (%), media ± desviación estándar sobre 3 semillas × instancias (MMR y el constructivo aleatorio son deterministas: solo instancias); en negrita, el mejor de los tres modelos en cada tamaño. La columna Módulo 2 es un único modelo, evaluado en las mismas instancias.

Variación entre semillas. Gap medio de cada uno de los modelos entrenados, resumido por arquitectura:

Tamaño	Base: media ± desv. entre semillas (rango)	A · sesgo MMR: media ± desv. entre semillas (rango)	B · ISAB: media ± desv. entre semillas (rango)
5×4	0,71 ± 0,36 (0,24–1,13)	1,00 ± 0,27 (0,62–1,21)	1,30 ± 0,58 (0,69–2,07)
10×8	2,33 ± 0,08 (2,24–2,42)	2,44 ± 0,30 (2,02–2,67)	2,21 ± 0,10 (2,09–2,33)
15×10	3,13 ± 0,14 (2,96–3,30)	3,40 ± 0,33 (3,07–3,85)	3,25 ± 0,29 (2,85–3,50)
20×15	1,96 ± 0,18 (1,75–2,19)	2,09 ± 0,19 (1,82–2,23)	2,01 ± 0,14 (1,90–2,21)
30×20	2,95 ± 0,16 (2,83–3,17)	2,97 ± 0,22 (2,81–3,28)	3,43 ± 0,53 (2,75–4,05)
50×40	2,01 ± 0,38 (1,48–2,30)	1,96 ± 0,20 (1,73–2,21)	1,86 ± 0,34 (1,39–2,18)
100×80	1,57 ± 0,51 (0,86–2,01)	1,52 ± 0,33 (1,11–1,93)	1,30 ± 0,44 (0,72–1,78)

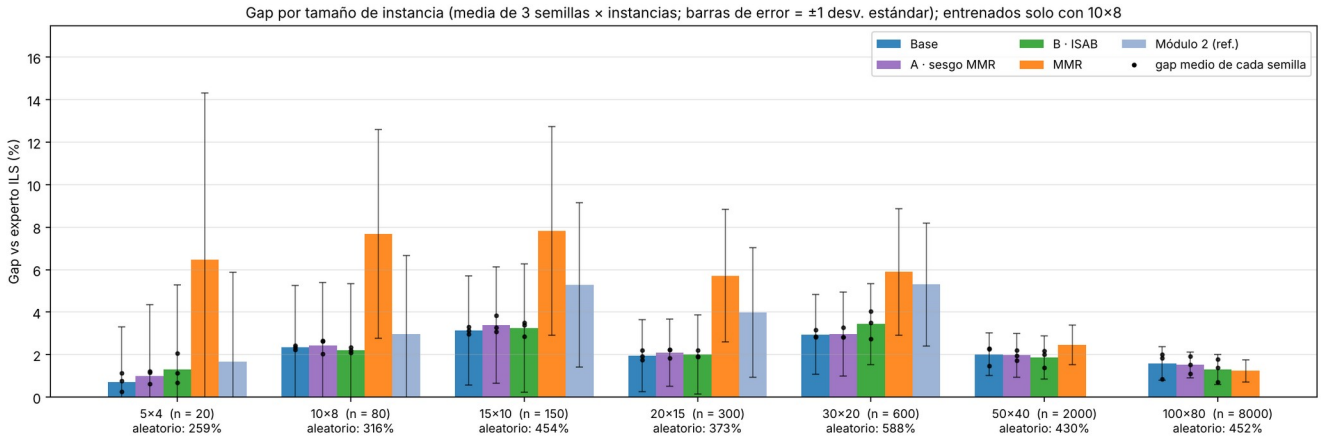


Figura 7. Gap medio por tamaño: barras = media de 3 semillas × instancias, barras de error = ±1 desviación estándar, puntos negros = gap medio de cada semilla.

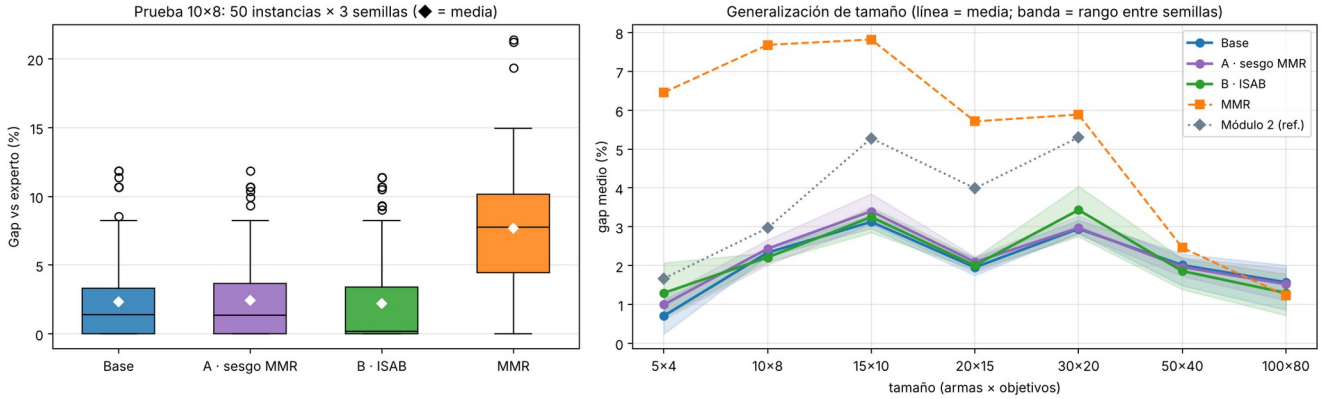


Figura 8. Izquierda: distribución del gap en las 50 instancias de prueba 10×8 × 3 semillas. Derecha: gap medio por tamaño (banda = rango entre semillas).

Método (prueba 10×8; 3 × 50 ejecuciones)	Gap medio	Desv.	Mediana	% óptimo	Gana a MMR
Base	2,33 %	2,93	1,41 %	32 %	79 %
A · sesgo MMR	2,44 %	2,96	1,35 %	31 %	79 %
B · ISAB	2,21 %	3,13	0,19 %	42 %	81 %
MMR	7,68 %	4,91	7,75 %	6 %	0 %
Módulo 2 (proyecto, 1 modelo)	2,97 %	3,68	1,85 %	30 %	72 %

¿Qué tan buena es la referencia en instancias grandes? Repetimos el ILS con 2 000 iteraciones y perturbación proporcional a W en los tres tamaños más grandes:

Tamaño	ILS largo mejora al ILS de referencia en	Base vs ILS largo	A · sesgo MMR vs ILS largo	B · ISAB vs ILS largo	MMR vs ILS largo
30×20	0,82 %	3,80 %	3,82 %	4,29 %	6,76 %
50×40	1,07 %	3,12 %	3,07 %	2,97 %	3,58 %
100×80	0,93 %	2,53 %	2,48 %	2,25 %	2,18 %

Tiempo de inferencia. Milisegundos por instancia, con el rollout por lotes en CPU (todos los hilos; modelos de la semilla 0):

Tamaño	Base	A · sesgo MMR	B · ISAB	MMR	Experto ILS
5×4	0,29	0,21	0,38	0,03	8,06
10×8	0,35	0,32	0,80	0,05	8,95
15×10	0,72	0,70	2,08	0,08	10,1
20×15	1,11	1,23	4,21	0,11	10,7
30×20	2,15	2,15	10,6	0,17	13,1
50×40	5,83	5,95	79,7	0,33	18,4
100×80	116	113	1.110	1,12	39,9

Análisis — tamaño de entrenamiento (10×8). Los tres modelos superan a MMR por al menos 5,2 pp (2,21–2,44 % frente a 7,68 %) y al modelo del módulo 2 (2,97 %), que además recibía features heurísticas. Sus medias son muy parecidas: la diferencia máxima (0,23 pp) es menor que la variación entre semillas de una misma arquitectura (hasta 0,65 pp, en A). La distribución sí cambia: B iguala al experto en el 42 % de las ejecuciones (base 32 %, A 31 %), su mediana es 0,19 % (base 1,41 %, A 1,35 %) y pierde contra MMR solo en el 3 % de los casos (base 7 %, A 10 %). Su media no mejora porque, cuando se equivoca, lo hace por montos parecidos a los de los otros modelos.

Generalización de tamaño. Entrenados solo con 10×8, los tres modelos se mantienen entre 0,7 % y 3,4 % hasta 30×20, mientras MMR queda entre 5,7 % y 7,8 %. Mejoran al módulo 2 en todos los tamaños compartidos: por 0,4–1,0 pp en 5×4 y 10×8, y por 1,9–2,4 pp en 15×10, 20×15 y 30×20, donde el modelo anterior se degradaba más.

Instancias grandes. En 50×40 y 100×80, MMR mejora mucho (2,46 % y 1,23 %) y la referencia se debilita: un ILS de 2 000 iteraciones mejora al de referencia en 1,07 % y 0,93 %. Todos los modelos entrenados superan a MMR en 50×40, pero en 100×80 MMR queda mejor que la media de las tres arquitecturas (1,30–1,57 %) y solo 3 de los 9 modelos lo superan (uno por arquitectura). Medido contra el ILS largo (tabla anterior), el orden de los métodos no cambia.

Diferencias entre arquitecturas. En cada tamaño, las medias de las tres arquitecturas difieren en a lo más 0,6 pp, del mismo orden que la variación entre semillas (desviación de 0,1 a 0,6 pp), así que ninguna domina. La base es la mejor en 4 de los 7 tamaños (5×4, 15×10, 20×15 y 30×20) y B en 3 (10×8, 50×40 y 100×80); en 100×80, B gana a MMR en el 52 % de las ejecuciones, contra 34–36 % de la base y A. A queda a ±0,3 pp de la base en todos los tamaños. B es la más sensible a la semilla en 5×4 y 30×20 (rangos 0,69–2,07 % y 2,75–4,05 %).

Tiempo. En 10×8 los modelos resuelven una instancia en 0,3–0,8 ms, entre 11 y 28 veces más rápido que el ILS (9,0 ms). Pero el rollout son W pasos de costo O(n·d): en 100×80 tarda 113–116 ms con la base y A, y 1,1 s con B, más que el ILS de 100 iteraciones (40 ms).

7. Conclusiones

- **Responsabilidades separadas.** El rediseño cumple la definición de la tarea: el encoder solo ve V_j y p_{ij} (token aprendible para las armas y la matriz como aristas) y corre una vez por instancia; el estado entra solo al decoder. Sin las features heurísticas del módulo 2, la base supera a MMR desde la primera época y mejora al módulo 2 en los 5 tamaños compartidos (2,33 % frente a 2,97 % en 10×8; 1,96–3,13 % frente a 3,99–5,30 % entre 15×10 y 30×20).
- **Complejidad.** El encoder es lineal en n = W·T (atención bipartita) y se paga una vez; el decoder es lineal por paso (pendiente medida 1,06 frente a 2,11 del módulo 2), así que una instancia 50×40 se resuelve 36 veces más rápido que con el modelo anterior. La interacción entre pares de la variante B sería cuadrática (pendiente 2,22) y con ISAB queda lineal (pendiente 1,20, o 1,02 sin el tamaño más grande). El límite que queda es el número de pasos: W pasos de O(n·d) por instancia, que en 100×80 ya es más lento que el ILS.
- **Sesgo inductivo (A).** Con la ganancia de MMR como sesgo, el modelo parte desde MMR (6,9 % en vez de 821 %), llega un poco antes a los umbrales (gap < 3 % en 9–12 épocas frente a 10–13) y los α aprendidos crecen, así que el modelo sí usa el criterio. Pero la base aprende lo mismo en una época y la calidad final es equivalente (diferencias de a lo más 0,3 pp, dentro de la variación entre semillas): en este problema, el sesgo acelera el inicio pero no cambia el resultado final.
- **Atención eficiente (B).** Que los pares interactúen con ISAB da la menor pérdida, la mayor accuracy y más soluciones exactas (42 % iguales al experto en 10×8 y solo 3 % de derrotas contra MMR), pero no un menor gap medio, y cuesta 4 veces más por época y hasta 10 veces más en inferencia.
- **Recomendación.** Con estos datos, la base es la mejor relación entre calidad y costo; A conviene si el presupuesto de entrenamiento es muy corto (parte desde MMR) y B si importa resolver exactamente más instancias. Como trabajo futuro, proponemos entrenar con varios tamaños (en 50×40 y 100×80 MMR se acerca) y combinar A y B.

Referencias

Ahuja, R. K., Kumar, A., Jha, K. C. y Orlin, J. B. (2007). Exact and heuristic algorithms for the weapon-target assignment problem. Operations Research, 55(6), 1136–1146.

den Broeder, G. G., Ellison, R. E. y Emerling, L. (1959). On optimum target assignments. Operations Research, 7(3), 322–326.

Kline, A., Ahner, D. y Hill, R. (2019). The weapon-target assignment problem. Computers & Operations Research, 105, 226–236.

Kool, W., van Hoof, H. y Welling, M. (2019). Attention, learn to solve routing problems! ICLR.

Kwon, Y.-D., Choo, J., Yoon, I., Park, M., Park, D. y Gwon, Y. (2021). Matrix encoding networks for neural combinatorial optimization. NeurIPS.

Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S. y Teh, Y. W. (2019). Set Transformer: a framework for attention-based permutation-invariant neural networks. ICML.

Ma, X., Kong, X., Wang, S., Zhou, C., May, J., Ma, H. y Zettlemoyer, L. (2021). Luna: Linear unified nested attention. NeurIPS.

Material del curso (TSP-Framework): notebooks de sesgo inductivo, atención lineal (LUNA, ISAB), esparsificación y GAT/EGAT.